Mata kuliah: Struktur data dan Algoritma - IF207
Nama Mahasiswa: Nalendro Agung Wicaksono
NIM: 250401020137

1. Diketahui *HashTable* kosong kapasitas N = 11 dan fungsi hash H(k) = k mod 11. Deretan kunci yang dimasukkan secara berurutan: [43, 22, 1, 12, 34, 56, 77, 88]
    a. Linear Probing, Jika terjadi tabrakan, cari slot kosong berikutnya dengan H(k, i) = (H(k) + i) (mod 11).

        - 43: 43 mod 11 = 10 => Indeks 10
        - 22: 22 mod 11 = 0 => Indeks 0
        - 1: 1 mod 11 = 1 => Indeks 1
        - 12: 12 mod 11 = 1 (Penuh). i=1 => (1+1) mod 11 = 2 => Indeks 2
        - 34: 34 mod 11 = 1 (Penuh). i=1,2 => (1+2) mod 11 = 3 => Indeks 3
        - 56: 56 mod 11 = 1 (Penuh). i=1,2,3 => (1+3) mod 11 = 4 => Indeks 4
        - 77: 77 mod 11 = 1 (Penuh). i=1..5 => (0+5) mod 11 = 5 => Indeks 5
        - 88: 88 mod 11 = 1 (Penuh). i=1..6 => (0+6) mod 11 = 6 => Indeks 6

Indeks	0	1	2	3	4	5	6	7	8	9	10
Key	22	1	12	34	56	77	88	-	-	-	43

Table Linear Probing

    b. Separate Chaining, Jika terjadi Tabrakan tambahkan kunci ke akhir linkedlist pada indeks
        - 43: 43 mod 11 = 10 → [43]
        - 22:22 mod 11 = 0 → [22]
        - 1: 1 mod 11 = 1 → [1]

- 12: 12 mod 11 = 1 → [1 => 12]
- 34: 34 mod 11 = 1 → [1 => 12 => 34]
- 56: 56 mod 11 = 1 → [1 => 12 => 34 => 56]
- 77: 77 mod 11 = 0 → [22 => 77]
- 88: 88 mod 11 = 0 → [22 => 77 => 88]

Indeks	Isi Linked List
0	22 => 77 => 88
1	1 => 12 => 34 => 56
2-9	-
10	43

Table Separate Chaining

2. Implementasi Program struktur data Hash Table dengan Java

```
public class HashTable<K, V> {

    static class HashNode<K, V> {
        K key;
        V value;
        HashNode<K, V> next;

        public HashNode(K key, V value) {
            this.key = key;
            this.value = value;
        }
    }

    private HashNode<K, V>[] chainTable;
    private int capacity;

    @SuppressWarnings("unchecked")
    public HashTable() {
        this.capacity = 10;
        this.chainTable = new HashNode[capacity];
    }

    private int getBucketIndex(K key) {
        int hashCode = key.hashCode();
        int index = hashCode % capacity;
        if (index < 0) {
            index += capacity;
        }
```

```
        return index;
    }

    public void insert(K key, V value) {
        int bucketIndex = getBucketIndex(key);
        HashNode<K, V> head = chainTable[bucketIndex];
        while (head != null) {
            if (head.key.equals(key)) {
                head.value = value;
                return;
            }
            head = head.next;
        }

        HashNode<K, V> newNode = new HashNode<>(key, value);
        head = chainTable[bucketIndex];
        if (head == null) {
            chainTable[bucketIndex] = newNode;
        } else {
            while (head.next != null) {
                head = head.next;
            }
            head.next = newNode;
        }
    }

    public String search(K key) {
        int bucketIndex = getBucketIndex(key);
        HashNode<K, V> head = chainTable[bucketIndex];

        while (head != null) {
            if (head.key.equals(key)) {
                return head.value.toString();
            }
            head = head.next;
        }
        return "Tidak ditemukan";
    }

    public void remove(K key) {
        int bucketIndex = getBucketIndex(key);
        HashNode<K, V> head = chainTable[bucketIndex];
        HashNode<K, V> prev = null;

        while (head != null) {
```

```
            if (head.key.equals(key)) {
                break;
            }
            prev = head;
            head = head.next;
        }

        if (head == null) {
            System.out.println("Kunci '" + key + "' tidak ditemukan untuk
dihapus.");
            return;
        }

        if (prev != null) {
            prev.next = head.next;
        } else {
            chainTable[bucketIndex] = head.next;
        }
        System.out.println("Kunci '" + key + "' berhasil dihapus.");
    }

    public void display() {
        for (int i = 0; i < capacity; i++) {
            System.out.print("Indeks " + i + ": ");
            HashNode<K, V> head = chainTable[i];
            if (head == null) {
                System.out.println("-");
            } else {
                while (head != null) {
                    System.out.print("[" + head.key + ": " + head.value + "]");
                    if (head.next != null) {
                        System.out.print(" -> ");
                    }
                    head = head.next;
                }
                System.out.println();
            }
        }
    }

    public static void main(String[] args) {
        HashTable<Integer, String> ht = new HashTable<>();
        System.out.println("=== TUGAS STRUKTUR DATA: HASH TABLE SEPARATE CHAINING
===");
```

```
        System.out.println("\n--- Melakukan Operasi Insert ---");
        ht.insert(43, "Data A");
        ht.insert(22, "Data B");
        ht.insert(1, "Data C");
        ht.insert(12, "Data D");
        ht.insert(34, "Data E");
        ht.insert(56, "Data F");
        ht.insert(77, "Data G");
        ht.insert(88, "Data H");
        ht.display();

        System.out.println("\n--- Melakukan Operasi Search ---");
        System.out.println("Cari kunci 12: " + ht.search(12));
        System.out.println("Cari kunci 34: " + ht.search(34));
        System.out.println("Cari kunci 99: " + ht.search(99));

        System.out.println("\n--- Melakukan Operasi Remove ---");
        ht.remove(12);
        ht.remove(22);
        ht.display();

        System.out.println("\n--- Search Setelah Remove ---");
        System.out.println("Cari kunci 12: " + ht.search(12));
    }
}
```

- Hasil program, hash table separate chaining setelah melakukan remove data (key:12) lalu menampilkan data (key:12) yang tidak ditemukan

```
➜  HashTable java HashTable
=== TUGAS STRUKTUR DATA: HASH TABLE SEPARATE CHAINING ===

--- Melakukan Operasi Insert ---
Indeks 0: -
Indeks 1: [1: Data C]
Indeks 2: [22: Data B] -> [12: Data D]
Indeks 3: [43: Data A]
Indeks 4: [34: Data E]
Indeks 5: -
Indeks 6: [56: Data F]
Indeks 7: [77: Data G]
Indeks 8: [88: Data H]
Indeks 9: -

--- Melakukan Operasi Search ---
Cari kunci 12: Data D
Cari kunci 34: Data E
Cari kunci 99: Tidak ditemukan

--- Melakukan Operasi Remove ---
Kunci '12' berhasil dihapus.
Kunci '22' berhasil dihapus.
Indeks 0: -
Indeks 1: [1: Data C]
Indeks 2: -
Indeks 3: [43: Data A]
Indeks 4: [34: Data E]
Indeks 5: -
Indeks 6: [56: Data F]
Indeks 7: [77: Data G]
Indeks 8: [88: Data H]
Indeks 9: -

--- Search Setelah Remove ---
Cari kunci 12: Tidak ditemukan
```

- Hasil program, hash table separate chaining setelah remove data yang dapat tidak dapat ditemukan (key:89), search data (key:34) setelah remove